

Connecting the Microsoft App Registration

DriverAI Hiring Agent (P1 tools + P2 worker) · Graph app-only authentication · written for a setup owner who does **not** hold Global Administrator · 15 September 2026

1. What the registration is for

P2 and three P1 helper tools talk to Microsoft Graph with **app-only authentication** — the OAuth `client_credentials` grant, confirmed in `HiringAgent_P2/sharepoint_client.py` and `HiringAgent_P1/trigger_reset.py`. There is no signed-in user; the process runs unattended and presents a tenant ID, a client ID and a client secret to `https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token` with scope `https://graph.microsoft.com/.default`.

Component	Needs the registration?
P1 Power Automate flow (live intake)	No. It authenticates through its three Power Automate connections — Office 365 Outlook, SharePoint, Excel Online — and holds no secret. It keeps running whatever happens here.
P2 scoring worker (<code>bot.py</code> , <code>function_app.py</code>)	Yes. Reads and writes the SharePoint workbook, downloads résumés, sends applicant and admin mail.
<code>trigger_reset.py</code> , <code>bulk_move_tool.py</code> , <code>audit_p1_live_state.py</code>	Yes. Move mail back to the Inbox to replay P1, and audit live state.

CURRENT STATE IN THIS WORKSPACE

An app registration named **DriverAI-HiringAgent-App** already exists, and `HiringAgent_P2/.env` holds live `TENANT_ID` , `CLIENT_ID` and `CLIENT_SECRET` values dated 2026-09-13. If Graph calls currently succeed, you need Section 6 (verify) and Section 8 (secret expiry) only. Sections 3–5 are for a new tenant, a re-registration, or a permission that was never consented.

2. The access problem, precisely

This is the part that is worth getting right before you ask anyone for anything, because the obvious ask is the wrong one.

The four permissions this system needs are Microsoft Graph **application permissions** (Microsoft calls them *app roles*). Per Microsoft's own documentation on granting tenant-wide admin consent, the roles that may consent to them are:

Directory role	Can grant consent for Microsoft Graph <i>application</i> permissions?
Global Administrator	Yes
Privileged Role Administrator	Yes — for any permission, for any API
Cloud Application Administrator	No — may consent to any API <i>except</i> Microsoft Graph app roles
Application Administrator	No — same exclusion
AI Administrator	No — same exclusion
Application Developer	No. Lets you <i>create</i> registrations, never consent to them
Owner of the app registration	No. Lets you manage secrets and request permissions, never consent to them

DO NOT ASK FOR THE WRONG ROLE

Asking for **Cloud Application Administrator** is the common mistake — it sounds like the right one, it is routinely handed out, and it will *not* let you press **Grant admin consent** on these four permissions. You will end up back in the same queue a week later. A custom directory role carrying the application-consent permission also works, but building one is more work for the admin than the ten minutes in Section 4.

So a one-time action by a Global Administrator or Privileged Role Administrator is unavoidable. What *is* avoidable is needing them again: everything else — creating the registration, rotating the secret, reading the IDs, filling in `.env`, testing — can be delegated to you permanently.

3. The two things to ask for

Ask 1 — permanent, no ongoing admin cost. Add me as an **Owner** of the `DriverAI-HiringAgent-App` registration. Entra admin centre → Applications → App registrations → the app → **Owners** → Add owner. This lets me rotate the expiring client secret and adjust the requested permission list myself, without coming back to you.

Ask 2 — one-time, about ten minutes. A Global Administrator or Privileged Role Administrator grants tenant-wide admin consent to the four Microsoft Graph application permissions in Section 4. No other role can do this. Section 4 is written to be forwarded to them as-is.

Possible Ask 3 — only if mail calls return 403. An **Exchange Administrator** scopes the Exchange application access policy to this app. See Section 7. This is a different role and a different admin centre; do not bundle it with Ask 2 unless the 403 has actually appeared.

If your tenant has the admin consent request workflow switched on (Entra ID → Enterprise applications → Consent and permissions → Admin consent settings), you can also raise the request in-product and let the designated reviewers approve it. Requests land in Entra ID → Enterprise applications → **Admin consent requests**. That workflow triggers on an interactive sign-in prompt, which an app-only daemon never produces, so treat it as a routing convenience — the reviewer still has to hold one of the two roles above.

4. For the administrator — the ten-minute part

Forward this section. It assumes Global Administrator or Privileged Role Administrator.

4a. Create the registration (skip if `DriverAI-HiringAgent-App` already exists)

1. Go to <https://entra.microsoft.com>.
2. **Applications** → **App registrations** → **+ New registration**.
3. Name it recognisably, for example `DriverAI-HiringAgent-App`.
4. Choose **Accounts in this organizational directory only** (single tenant).
5. Leave Redirect URI empty. Select **Register**.
6. Copy the **Directory (tenant) ID** and the **Application (client) ID**.
7. **Certificates & secrets** → **+ New client secret**. Give it a description and an expiry, then **Add**.

COPY THE VALUE, NOT THE SECRET ID

The two columns sit side by side and look almost identical. The **Value** is displayed once. Refresh the page and it is masked permanently — the only remedy is to delete the secret and create another. Put the expiry date in a shared calendar before you leave the page.

4b. Add the permissions and consent to them

1. In the registration, choose **API permissions**.
2. **+ Add a permission** → **Microsoft Graph**.
3. Choose **Application permissions** — *not* Delegated permissions.
4. Select all four in the table below, then **Add permissions**.
5. Choose **Grant admin consent for <organisation>** and confirm.
6. Check that every row shows a green tick in the **Status** column. A row without one is not consented, whatever else the page says.

Permission	Type	Why the system needs it
<code>Sites.ReadWrite.All</code>	Application	Read and update the candidate workbook on SharePoint
<code>Files.ReadWrite.All</code>	Application	Read, rename and move résumé files
<code>Mail.Send</code>	Application	Send P2's applicant replies and admin alerts
<code>Mail.ReadWrite</code>	Application	Move mail back to the Inbox so P1 can be replayed

APPLICATION, NOT DELEGATED

Delegated permissions act on behalf of a signed-in person. There is no signed-in person here — the worker runs unattended on a schedule. Pick Delegated by mistake and nothing authenticates, and the resulting error text points nowhere near the cause.

4c. If you prefer a direct consent link

The portal buttons open a standard consent prompt that can be reached by URL:

```
https://login.microsoftonline.com/{tenant-id}/adminconsent?client_id={client-id}
```

Substitute the Directory (tenant) ID — or any verified domain of the tenant — and the Application (client) ID. This endpoint redirects to a registered redirect URI on completion, so if the registration has none (step 4a leaves it empty), add one first or simply use the portal button in 4b, which needs no such setup.

5. Handing the values back

Three values come out of Section 4: tenant ID, client ID, client secret Value. Only the secret is sensitive; the two IDs are not.

TRANSPORT OF THE SECRET

Do not send the secret by email or chat, and do not put it in a shared folder or a ticket. Use a password manager's one-time share link, or have the administrator paste it directly into the target `.env` or Function App setting. Anyone holding it can read and write the whole SharePoint site and send mail as the shared mailbox.

Local deployment — `HiringAgent_P2\.env`, which is already in `.gitignore`:

```
TENANT_ID=<Directory (tenant) ID>
CLIENT_ID=<Application (client) ID>
CLIENT_SECRET=<the Value, not the Secret ID>
SENDER_MAILBOX=apply@driverai.io
```

Azure Functions deployment — the same three keys go into the Function App's **Application settings** (see `function_app.py`), alongside `SHAREPOINT_HOSTNAME`, `SHAREPOINT_SITE_PATH`, `SHAREPOINT_TABLE` and the rest. Never commit either form.

6. Verifying the connection

Run these in order. Each one fails differently, which is what makes them useful.

Check 1 — the secret is valid (does a token come back?)

```
curl -s -X POST \
  "https://login.microsoftonline.com/$TENANT_ID/oauth2/v2.0/token" \
  -d "grant_type=client_credentials" \
  -d "client_id=$CLIENT_ID" \
  -d "client_secret=$CLIENT_SECRET" \
  -d "scope=https://graph.microsoft.com/.default"
```

An `access_token` in the response means tenant ID, client ID and secret are all correct. This step succeeds even when *no* permission has been consented — a token is issued regardless, which is exactly why a green result here proves less than it appears to.

Check 2 — the permissions were actually consented

Paste the token's payload (the middle dot-separated segment, base64) into any JWT decoder and read the `roles` claim. All four permissions must be listed. A token with **no `roles` claim at all** is the signature of permissions added but never consented — go back to step 5 of Section 4b. Alternatively, call a real endpoint:

```
curl -s -H "Authorization: Bearer $TOKEN" \
  "https://graph.microsoft.com/v1.0/sites/$SHAREPOINT_HOSTNAME:/sites/$SITE_PATH"
```

Check 3 — the end-to-end path

Run `audit_p1_live_state.py` from `HiringAgent_P1/`, which reads the same `.env` and exercises the mailbox and SharePoint calls the live system makes. Keep `HIRING_SUPPRESS_EMAILS` set until you have deliberately decided to go live — that flag is what stops a verification run from mailing real applicants.

7. Failures and what each one means

Symptom	Cause and fix
AADSTS700016 — application not found in directory	Wrong <code>CLIENT_ID</code> , or the right client ID against the wrong tenant. Confirm both against the registration's Overview page.
AADSTS7000215 — invalid client secret	The Secret ID was copied instead of the Value , or the secret has expired. Both look identical from the application's side. Create a new secret (Section 8).
AADSTS900023 / tenant not found	<code>TENANT_ID</code> is wrong or has stray whitespace. The code strips it, but a wrapped line in <code>.env</code> is a common source.
Token issued, but Graph returns 403 Authorization_RequestDenied	Admin consent was never granted, or was granted to Delegated rather than Application permissions. Decode the token — an absent <code>roles</code> claim confirms it. Back to Section 4b.
Mail calls return 403 ErrorAccessDenied despite consent showing green	Not a Graph permission problem. An Exchange Online application access policy is restricting which mailboxes this app may touch. See below.
Everything worked, then stopped on a date nobody changed anything	The client secret expired. This is the single most common way the system dies quietly.
The administrator cannot see Grant admin consent , or it is greyed out	They hold Cloud Application Administrator or Application Administrator, which cannot consent to Graph app roles. Section 2 explains; escalate to Global Administrator or Privileged Role Administrator.

The Exchange application access policy

Graph application permissions like `Mail.Send` are tenant-wide by default — every mailbox. Many tenants therefore constrain them with an application access policy, which produces a `403 ErrorAccessDenied` on every mail call while the Entra permissions page shows four cheerful green ticks. This condition occurred on this project before and was resolved on 15 July 2026. It never affected the P1 flow, which authenticates through its own Power Automate connections.

The fix requires an **Exchange Administrator** in Exchange Online PowerShell, not an Entra administrator:

```
# inspect what is currently in force
Get-ApplicationAccessPolicy

# allow this app to reach the shared mailbox only
New-ApplicationAccessPolicy -AppId <CLIENT_ID> `
  -PolicyScopeGroupId apply@driverai.io `
  -AccessRight RestrictAccess `
  -Description "DriverAI Hiring Agent - apply mailbox only"
```

Scoping the policy this way is worth doing even when no 403 has appeared: it narrows a tenant-wide mail permission down to the one mailbox the system actually uses, which usually makes the original consent an easier decision for a cautious administrator.

8. Keeping it alive

- **Diary the secret expiry the day it is created.** Expiry is silent, it arrives months later, and the failure looks like a code fault rather than a certificate one. A reminder two weeks ahead is enough.

- **Rotate as an Owner, not as an admin.** With Ask 1 granted, you create the replacement secret yourself: add the new secret, update `.env` or the Function App settings, verify with Check 1, then delete the old one. Consent is *not* re-required — rotating a secret does not touch the consent grant, so the administrator is not involved.
- **Adding a fifth permission later means a fresh consent.** Any new application permission re-opens Ask 2. Add what you need in one pass rather than discovering a gap a month on.
- **Record who holds the roles.** Note which named person granted consent and which holds Exchange Administrator. When this breaks it will be months from now, and the first hour will otherwise go on finding them again.

Sources: `HiringAgent_P1/docs/P1_RUNBOOK.md` steps 8–9 · `HiringAgent_P1/docs/README.md` · `P1_Technical_Design_Document_Consolidated.md` · `HiringAgent_P2/sharepoint_client.py`, `function_app.py`, `bot.py` · Microsoft Learn, “Grant tenant-wide admin consent to an application”, updated 27 August 2026.